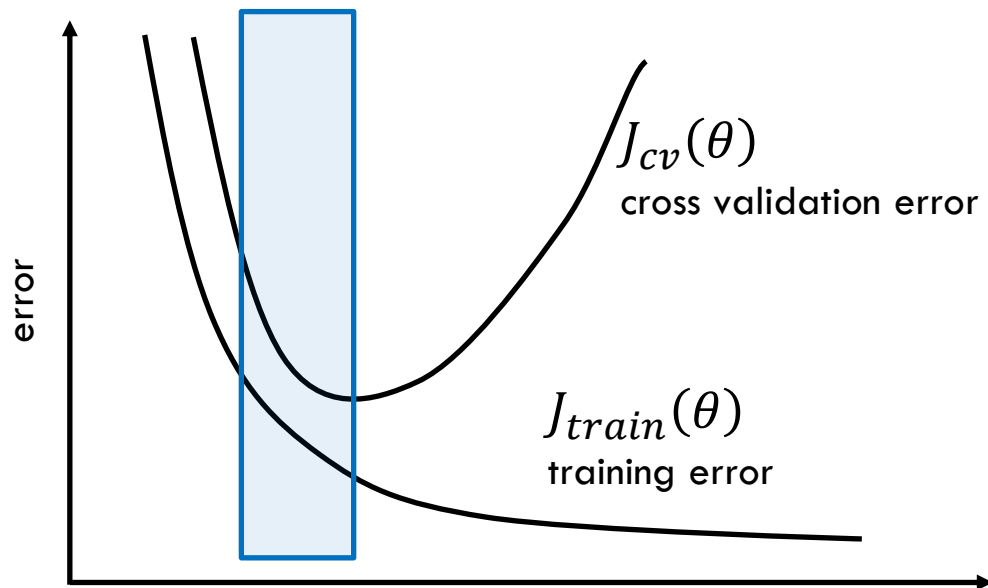
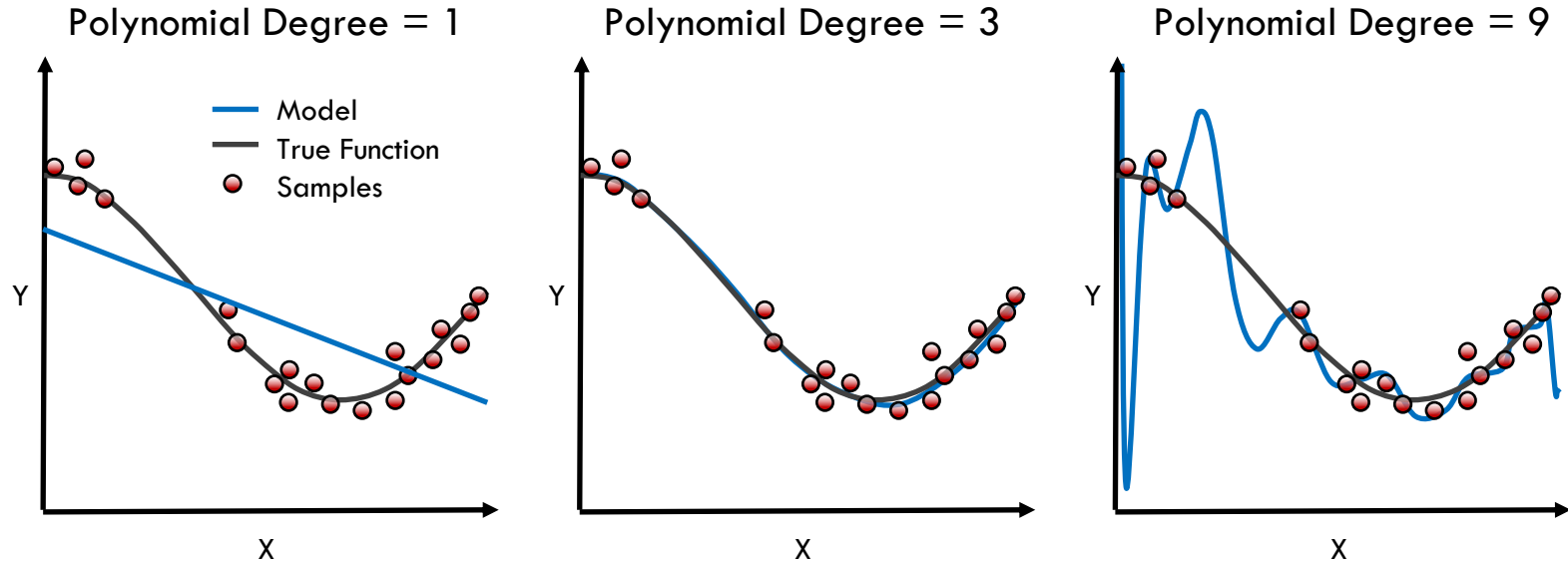


Regularization and Feature Selection

Model Complexity vs Error

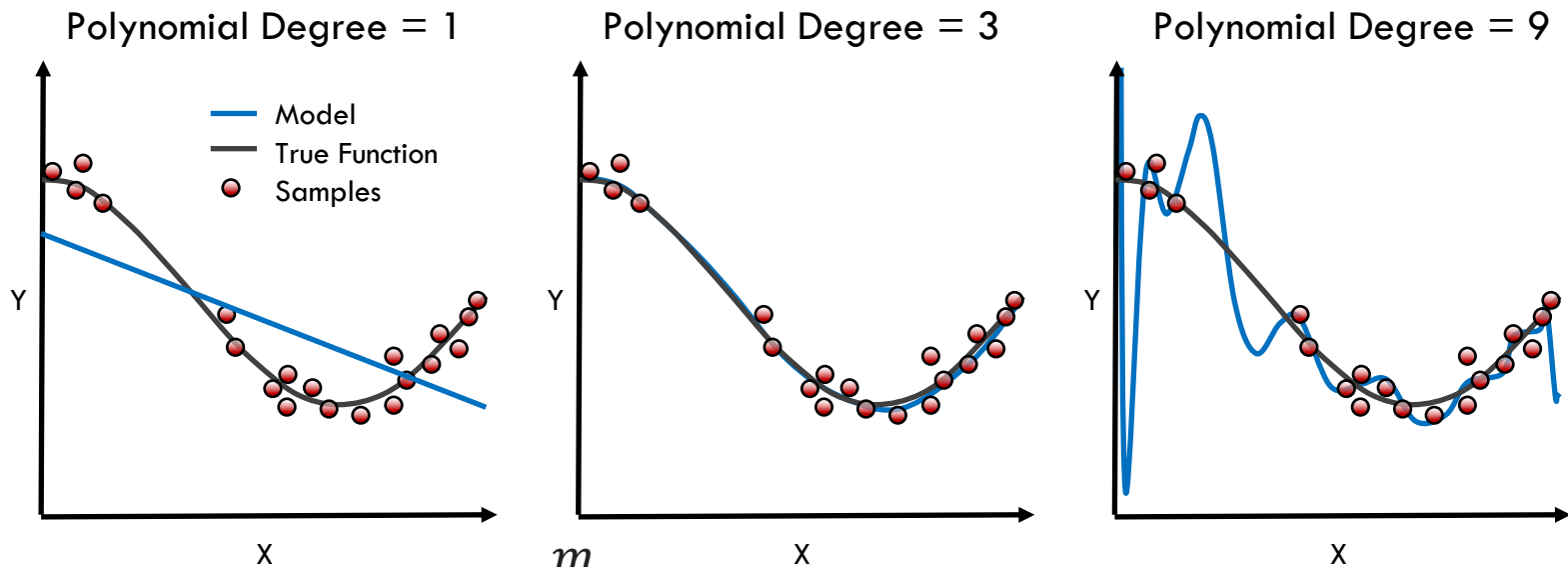


Preventing Under- and Overfitting



- How to use a degree 9 polynomial and prevent overfitting?

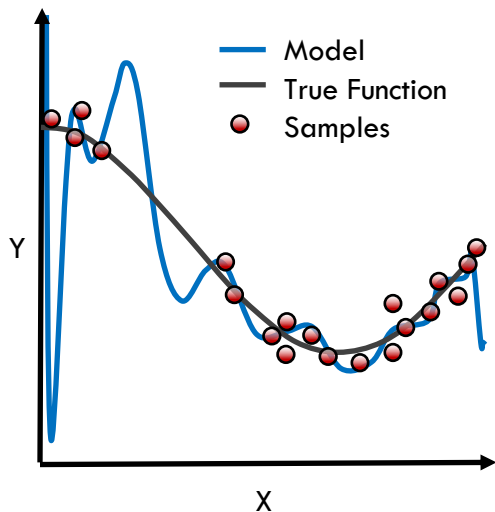
Preventing Under- and Overfitting



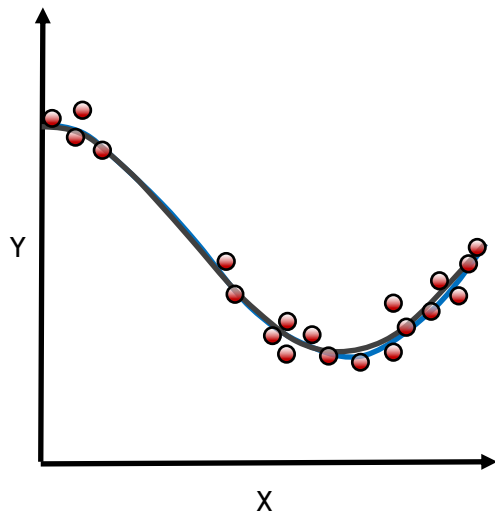
$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Regularization

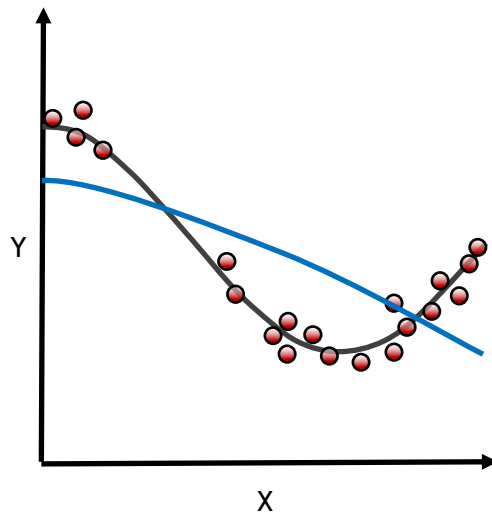
Poly Degree=9, $\lambda=0.0$



Poly Degree=9, $\lambda=1e-5$



Poly Degree=9, $\lambda=0.1$



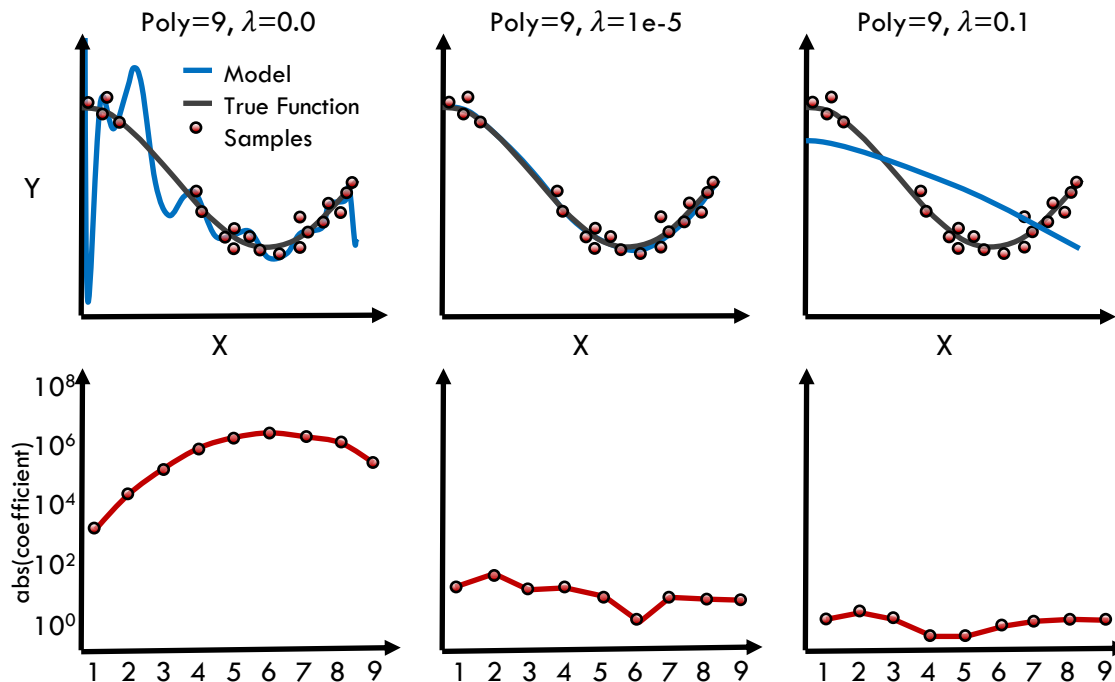
$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2 + \lambda \sum_{j=1}^k \beta_j^2$$

Ridge Regression (L2)

$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2 + \lambda \sum_{j=1}^k \beta_j^2$$

- Penalty shrinks magnitude of all coefficients
- Larger coefficients strongly penalized because of the squaring

Effect of Ridge Regression on Parameters



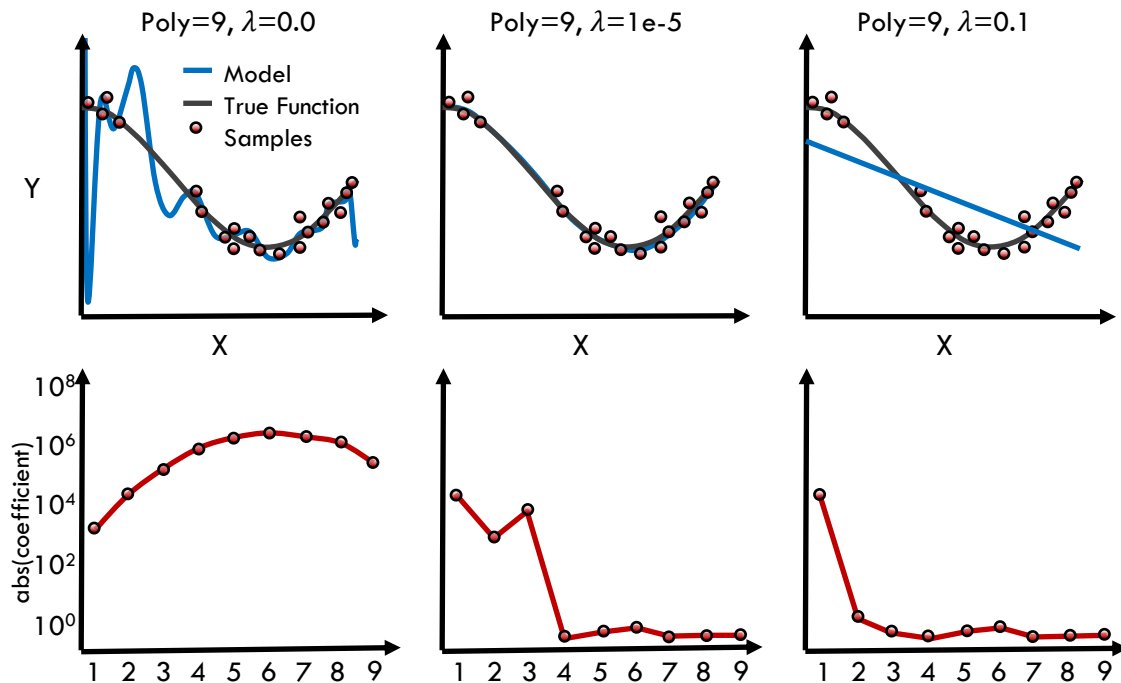
$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2 + \lambda \sum_{j=1}^k \beta_j^2$$

Lasso Regression (L1)

$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2 + \lambda \sum_{j=1}^k |\beta_j|$$

- Penalty selectively shrinks some coefficients
- Can be used for feature selection
- Slower to converge than Ridge regression

Effect of Lasso Regression on Parameters



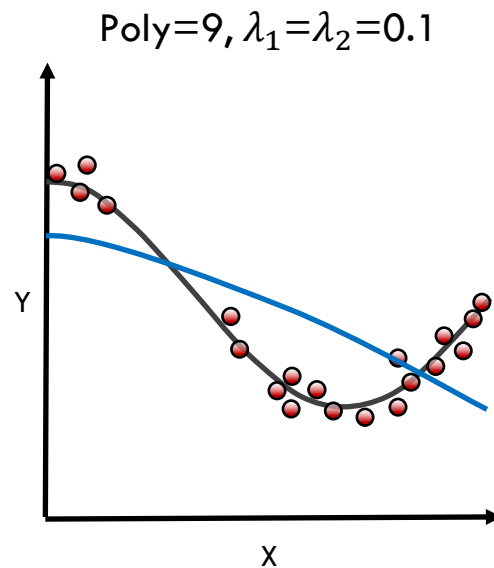
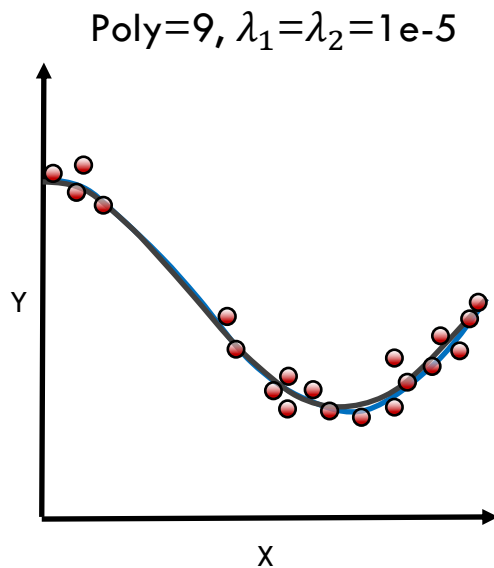
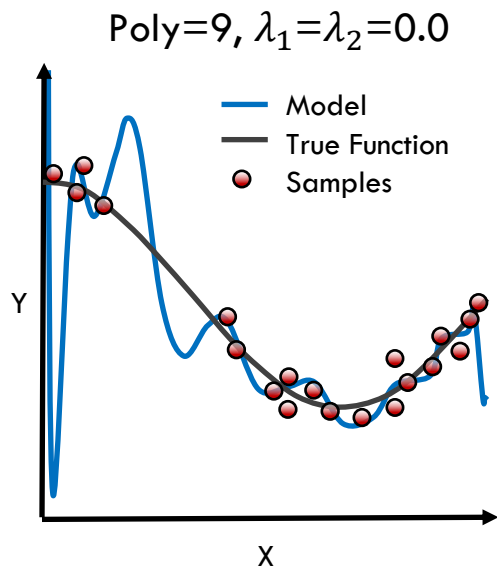
$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2 + \lambda \sum_{j=1}^k |\beta_j|$$

Elastic Net Regularization

$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2 + \lambda_1 \sum_{j=1}^k |\beta_j| + \lambda_2 \sum_{j=1}^k \beta_j^2$$

- Compromise of both Ridge and Lasso regression
- Requires tuning of additional parameter that distributes regularization penalty between L1 and L2

Elastic Net Regularization



$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left((\beta_0 + \beta_1 x_{obs}^{(i)}) - y_{obs}^{(i)} \right)^2 + \lambda_1 \sum_{j=1}^k |\beta_j| + \lambda_2 \sum_{j=1}^k \beta_j^2$$

Hyperparameters and Their Optimization

- Regularization coefficients (λ_1 and λ_2) are empirically determined

Use Test Data to Tune λ ?

0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis Lawrence	PG-13	146
1	2013-05-03	Iron Man 3	200000000	409013994	Shane Black	PG-13	129
2	2013-11-22	Frozen	150000000	400738009	Chris BuckJennifer Lee	PG	108
3	2013-07-03	Despicable Me 2	76000000	368061265	Pierre CoffinChris Renaud	PG	98
4	2013-06-14	Man of Steel	225000000	291045518	Zack Snyder	PG-13	143
5	2013-10-04	Gravity			James Cameron	PG-13	91
6	2013-06-21	Monsters University			Byron Howard	G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug			J.R. Hartless	PG-13	161
8	2013-05-24	Fast & Furious 6	160000000	238679850	Justin Lin	PG-13	130
9	2013-03-08	Oz The Great and Powerful	215000000	234911825	Sam Raimi	PG	127
10	2013-05-16	Star Trek Into Darkness	190000000	228778661	J.J. Abrams	PG-13	123
11	2013-11-08	Thor: The Dark World	170000000	206362140	Alan Taylor	PG-13	120
12	2013-06-21	World War Z	190000000	202359711	Marc Forster	PG-13	116
13	2013-03-22	The Croods	135000000	187168425	Kirk De MiccoChris Sanders	PG	98
14	2013-06-28	The Heat			Matthew Vaughn	R	117
15	2013-06-07	We're the Millers			Mark Waters	R	110
16	2013-12-13	American Hustle			David O. Russell	R	138
17	2013-05-10	The Great Gatsby	105000000	144840419	Baz Luhrmann	PG-13	143

Hyperparameters and Their Optimization

- Regularization coefficients (λ_1 and λ_2) are empirically determined
- Want value that generalizes—do not use test data for tuning

Use Test Data to Tune λ ?

0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis	PG-13	146
1	2013-05-03	The Hunger Games: Mockingjay - Part 1	200000000	409013994	Shirley	PG-13	129
2	2013-11-22	Frozen	150000000	400738009		PG	108
3	2013-07-03	Despicable Me 3	76000000	368061265		PG	98
4	2013-06-14	Man of Steel	250000000	291045518		PG-13	143
5	2013-10-04	Gravity			Caron	PG-13	91
6	2013-06-21	Monsters University				G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug				PG-13	161
8	2013-05-24	Fast & Furious 6				PG-13	130
9	2013-03-08	Oz The Great and Powerful				PG	127
10	2013-05-16	Star Trek Into Darkness				PG-13	123
11	2013-11-08	Thor: The Dark World				PG-13	120
12	2013-06-21	World War Z				PG-13	116
13	2013-03-22	The Croods				PG	98
14	2013-06-28	The Heat				R	117
15	2013-08-07	We're the Millers			Thurber	R	110
16	2013-12-13	American Hustle				R	138
17	2013-05-10	The Hunger Games: Mockingjay - Part 2	105000000	144840419		PG-13	143

Hyperparameters and Their Optimization

- Regularization coefficients (λ_1 and λ_2) are empirically determined
- Want value that generalizes—do not use test data for tuning
- Create additional split of data to tune hyperparameters—validation set

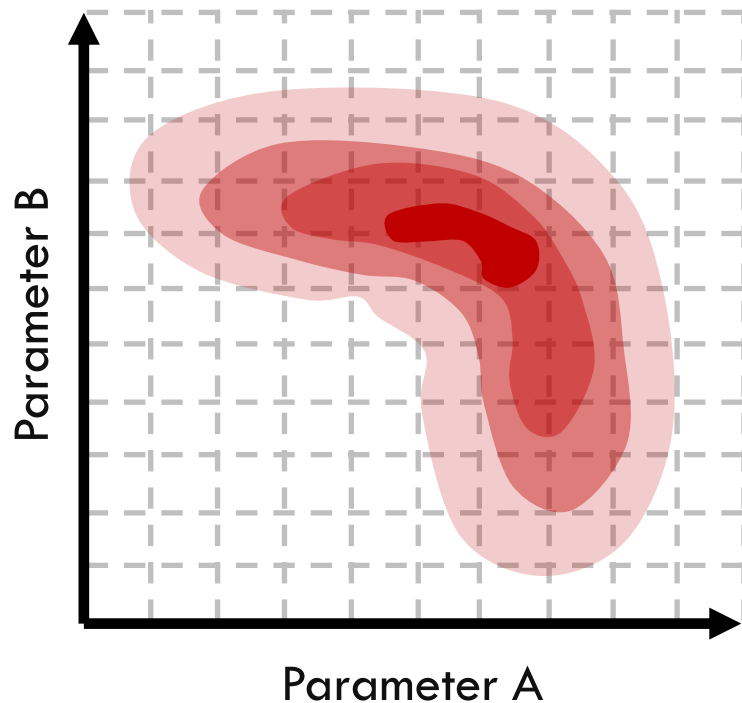
Tune λ with Cross Validation

0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis Lawrence	PG-13	146
1	2013-05-03	Iron Man 3	200000000	409013994	Shane Black	PG-13	129
2	2013-11-22	Frozen	150000000	100000000	Chris Buck Jennifer Lee	PG	108
3	2013-07-03	Despicable Me 2			Chris Renaud	PG	98
4	2013-06-14	Man of Steel				PG-13	143
5	2013-10-04	Gravity				PG-13	91
6	2013-06-21	Monsters University	NaN	268492764	Dan Scanlon	G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug	NaN	258366855	Peter Jackson	PG-13	161
8	2013-05-24	Fast & Furious 6	160000000	238679850	Justin Lin	PG-13	130
9	2013-03-08	Oz The Great and Powerful				PG	127
10	2013-05-16	Star Trek Into Darkness				PG-13	123
11	2013-11-08	Thor: The Dark World				PG-13	120
12	2013-06-21	World War Z	130000000	1202350713	Mark Forster	PG-13	116
13	2013-03-22	The Croods	135000000	187168425	Kirk De MiccoChris Sanders	PG	98
14	2013-06-28	The Heat				R	117
15	2013-08-07	We're the Millers			Will Thurbur	R	110
16	2013-12-13	American Hustle				R	138
17	2013-05-10	The Great Gatsby	105000000	144840419	Baz Luhrmann	PG-13	143

Grid Search

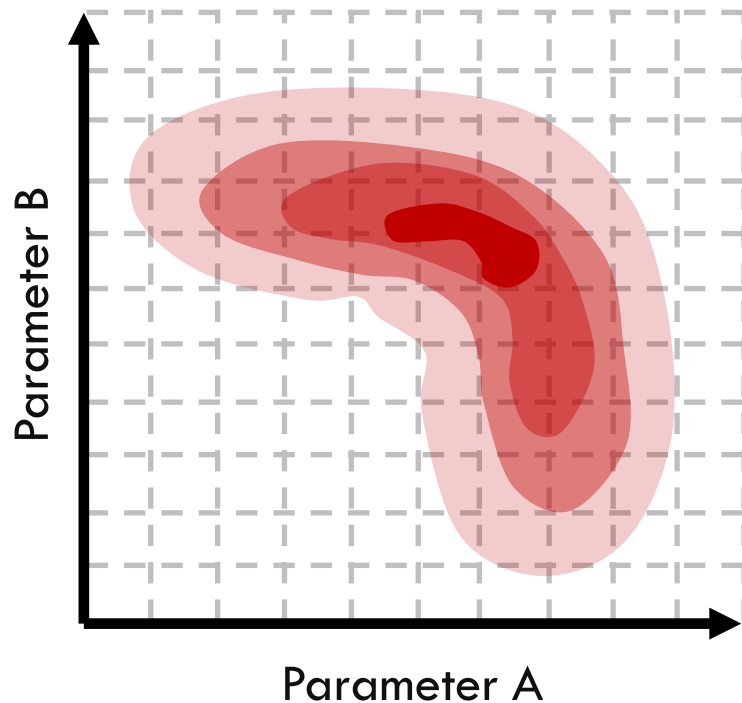
Generalized Hyperparameter Grid Search

- Hyperparameter selection for regularization / better models requires cross validation on training data
- Linear and logistic regression methods have classes devoted to grid search (e.g. LassoCV)



Generalized Hyperparameter Grid Search

- Grid search can be useful for other methods too, so a generalized method is desirable
- Scikit-learn contains GridSearchCV, which performs a grid search with parameters using cross validation



Grid Search with Cross Validation: The Syntax

Import the class containing the grid search method

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import GridSearchCV
```

Grid Search with Cross Validation: The Syntax

Import the class containing the grid search method

```
from sklearn.linear_model import LogisticRegression  
from sklearn.model_selection import GridSearchCV
```

Create an instance of the estimator and grid search class

```
LR = LogisticRegression(penalty='l2')  
GS = GridSearchCV(LR, param_grid={'c':[0.001, 0.01, 0.1]},  
                    scoring='accuracy', cv=4)
```

Grid Search with Cross Validation: The Syntax

Import the class containing the grid search method

```
from sklearn.linear_model import LogisticRegression  
from sklearn.model_selection import GridSearchCV
```

Create an instance of the estimator and grid search class

```
LR = LogisticRegression(penalty='l2')  
GS = GridSearchCV(LR, param_grid={'c':[0.001, 0.01, 0.1]},  
                  scoring='accuracy', cv=4)
```

logistic regression
method



Grid Search with Cross Validation: The Syntax

Import the class containing the grid search method

```
from sklearn.linear_model import LogisticRegression  
from sklearn.model_selection import GridSearchCV
```

Create an instance of the estimator and grid search class

```
LR = LogisticRegression(penalty='l2')  
GS = GridSearchCV(LR, param_grid={'c':[0.001, 0.01, 0.1]},  
                  scoring='accuracy', cv=4)
```

Fit the instance on the data to find the best model and then predict

```
GS = GS.fit(X_train, y_train)  
y_train = GS.predict(X_test)
```

Ridge Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Ridge
```

Ridge Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Ridge
```

Create an instance of the class

```
RR = Ridge(alpha=1.0)
```

Ridge Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Ridge
```

Create an instance of the class

```
RR = Ridge(alpha=1.0)
```



regularization
parameter

Ridge Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Ridge
```

Create an instance of the class

```
RR = Ridge(alpha=1.0)
```

Fit the instance on the data and then predict the expected value

```
RR = RR.fit(X_train, y_train)  
y_predict = RR.predict(X_test)
```

Ridge Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Ridge
```

Create an instance of the class

```
RR = Ridge(alpha=1.0)
```

Fit the instance on the data and then predict the expected value

```
RR = RR.fit(X_train, y_train)  
y_predict = RR.predict(X_test)
```

The `RidgeCV` class will perform cross validation on a set of values for alpha.

Lasso Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Lasso
```

Create an instance of the class

```
LR = Lasso(alpha=1.0)
```

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)  
y_predict = LR.predict(X_test)
```

The `LassoCV` class will perform cross validation on a set of values for alpha.

Lasso Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import Lasso
```

Create an instance of the class

```
LR = Lasso(alpha=1.0)
```



regularization
parameter

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)  
y_predict = LR.predict(X_test)
```

The **LassoCV** class will perform cross validation on a set of values for alpha.

Elastic Net Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import ElasticNet
```

Create an instance of the class

```
EN = ElasticNet(alpha=1.0, l1_ratio=0.5)
```

Fit the instance on the data and then predict the expected value

```
EN = EN.fit(X_train, y_train)  
y_predict = EN.predict(X_test)
```

The ElasticNetCV class will perform cross validation on a set of values for l1_ratio and alpha.

Elastic Net Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import ElasticNet
```

Create an instance of the class

```
EN = ElasticNet(alpha=1.0, l1_ratio=0.5)
```



alpha is the
regularization
parameter

Fit the instance on the data and then predict the expected value

```
EN = EN.fit(X_train, y_train)  
y_predict = EN.predict(X_test)
```

The **ElasticNetCV** class will perform cross validation on a set of values for **l1_ratio** and **alpha**.

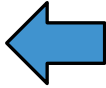
Elastic Net Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import ElasticNet
```

Create an instance of the class

```
EN = ElasticNet(alpha=1.0, l1_ratio=0.5)
```

 l1_ratio
distributes alpha
to L1/L2

Fit the instance on the data and then predict the expected value

```
EN = EN.fit(X_train, y_train)  
y_predict = EN.predict(X_test)
```

The **ElasticNetCV** class will perform cross validation on a set of values for l1_ratio and alpha.

Feature Selection

- Regularization performs feature selection by shrinking the contribution of features
- For L1-regularization, this is accomplished by driving some coefficients to zero
- Feature selection can also be performed by removing features

Why is Feature Selection Important?

- Reducing the number of features is another way to prevent overfitting (similar to regularization)
- For some models, fewer features can improve fitting time and/or results
- Identifying most critical features can improve model interpretability

Recursive Feature Elimination: The Syntax

Import the class containing the feature selection method

```
from sklearn.feature_selection import RFE
```

Create an instance of the class

```
rfeMod = RFE(est, n_features_to_select=5)
```

Fit the instance on the data and then predict the expected value

```
rfeMod = rfeMod.fit(X_train, y_train)  
y_predict = rfeMod.predict(X_test)
```

The `RFECV` class will perform feature elimination using cross validation.

Recursive Feature Elimination: The Syntax

Import the class containing the feature selection method

```
from sklearn.feature_selection import RFE
```

Create an instance of the class

```
rfeMod = RFE(est, n_features_to_select=5)
```

Fit the instance on the data and then predict the expected value

```
rfeMod = rfeMod.fit(X_train, y_train)  
y_predict = rfeMod.predict(X_test)
```

The `RFECV` class will perform feature elimination using cross validation.